

USER PROMPT:

You are a senior backend engineer. A junior developer asked you how to optimize SQL queries for better performance. Follow the Skeleton of Thought approach:

Step 1: Generate only the skeleton of your answer in 3–5 concise bullet points.
Step 2: Expand each bullet point into a clear and detailed explanation with examples.
Make sure the final answer is structured and easy to follow.

LLM RESPONSE:

Skeleton of Thought: SQL Query Optimization

- * **Indexing Strategy:** Proper indexing is crucial for efficient data retrieval.
- * **Query Rewriting Techniques:** How to rephrase queries for better execution plans.
- * **Database Schema Design:** The impact of schema on query performance.
- * **Execution Plan Analysis:** Understanding how the database executes queries.

Optimized SQL Query Performance for Junior Developers

As a senior backend engineer, I'm happy to guide you on optimizing SQL queries. Performance is key to a responsive and scalable application, and understanding how to make your queries run faster is a fundamental skill. We'll cover the core areas using a structured approach.

1. Indexing Strategy

Explanation: Think of indexes like the index in a book. Instead of reading every single page (row) to find information, an index allows the database to quickly locate the specific rows you need. Without appropriate indexes, the database often has to perform a "full table scan," which can be very slow, especially on large tables.

Key Considerations:

- * **Identify Frequently Queried Columns:** Columns used in `WHERE` clauses, `JOIN` conditions, `ORDER BY` clauses, and `GROUP BY` clauses are prime candidates for indexing.
- * **Choose the Right Index Type:**
 - * **B-tree Indexes:** The most common type, suitable for equality checks (`=`), range queries (`>`, `<`, `BETWEEN`), and sorting.
 - * **Hash Indexes:** Excellent for equality checks (`=`), but not for range queries or sorting.
 - * **Full-text Indexes:** For searching within text fields.
- * **Composite Indexes:** If you frequently query on multiple columns together (e.g., `WHERE last\_name = 'Smith' AND first\_name = 'John'`), a composite index on `(last\_name, first\_name)` can be more efficient than two separate indexes. The order of columns in a composite index matters.
- * **Avoid Over-Indexing:** While indexes improve read performance, they add overhead to write operations (inserts, updates, deletes) because the index also needs to be updated. Too many indexes can slow down your writes and consume disk space.
- * **Regularly Review and Maintain Indexes:** Indexes can become fragmented or outdated. Database systems have tools to analyze and rebuild indexes.

Example:

Let's say you have a `users` table with millions of records and you frequently run this query:

```
```sql
SELECT * FROM users WHERE email = 'john.doe@example.com';
```
```

Without an index on the `email` column, the database would scan every row. Adding an index:

```
```sql
CREATE INDEX idx_users_email ON users (email);
```
```

This will dramatically speed up that query. If you also frequently search by `username`, and sometimes filter by both `email` and `username` in the same query:

```
```sql
SELECT * FROM users WHERE email = 'john.doe@example.com' AND username = 'johnd';
```
```

A composite index might be beneficial:

```
```sql
CREATE INDEX idx_users_email_username ON users (email, username);
```
```

2. Query Rewriting Techniques

****Explanation:**** Sometimes, the way a query is written can lead the database optimizer to choose an inefficient execution plan. Rewriting the query, even if it achieves the same logical result, can sometimes yield significant performance gains.

****Key Techniques:****

- * ****`SELECT` Specific Columns:**** Avoid `SELECT \*`. Only retrieve the columns you actually need. This reduces the amount of data read from disk, transferred over the network, and processed by the application.
- * ****Use `JOIN`s Effectively:****
 - * ****Prefer `INNER JOIN` over `WHERE` clause joins:**** While often functionally equivalent, `INNER JOIN` syntax can be more readable and sometimes hints to the optimizer for better plans.
 - * ****Join on Indexed Columns:**** Ensure that the columns used in your `JOIN` conditions are indexed.
 - * ****Filter Early:**** Apply `WHERE` conditions as early as possible, ideally before joining tables, to reduce the number of rows being processed in subsequent steps.
 - * ****Avoid Functions in `WHERE` Clauses on Indexed Columns:**** Applying a function to a column in a `WHERE` clause (e.g., `WHERE YEAR(order\_date) = 2023`) often prevents the database from using an index on that column. Instead, rewrite it to compare the column to a value (e.g., `WHERE order\_date BETWEEN '2023-01-01' AND '2023-12-31'`).
 - * ****Use `EXISTS` vs. `COUNT(\*)` for Existence Checks:**** If you only need to know if *any* rows match a condition, `EXISTS` is generally more efficient than `COUNT(\*)`, as `EXISTS` can stop searching as soon as it finds the first match.
 - * ****Consider `UNION ALL` vs. `UNION`:**** `UNION` removes duplicate rows, which requires sorting and comparison, making it slower. If you know there are no duplicates or you don't care about them, `UNION ALL` is faster.
 - * ****Subqueries vs. Joins:**** Modern database optimizers are very good at handling subqueries. However, in some cases, a join might be more performant. Analyze the execution plan to see which works better for your specific scenario.

****Example:****

Consider a query to find customers who have placed an order in the last 30 days:

****Inefficient:****

```
```sql
SELECT c.customer_name
FROM customers c
WHERE c.customer_id IN (SELECT o.customer_id FROM orders o WHERE o.order_date >= DATE('now', '-30 days'));
```
```

****More Efficient (using JOIN and filtering early):****

```
```sql
SELECT DISTINCT c.customer_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= DATE('now', '-30 days');
```
```

(Note: `DISTINCT` is used here because a customer might have multiple orders. If you only need the customer name once, this is better than the subquery version which might scan all orders and then do a lookup for each customer ID.)

Alternatively, if you don't need the `customer\_name` and just want to know *if* a customer has placed an order:

****Using EXISTS (efficient for existence checks):****

```
```sql
SELECT c.customer_name
FROM customers c
WHERE EXISTS (
 SELECT 1
 FROM orders o
 WHERE o.customer_id = c.customer_id
 AND o.order_date >= DATE('now', '-30 days')
);
```
```

3. Database Schema Design

****Explanation:**** The way your tables are structured (your schema) has a profound impact on query performance. A well-designed schema can make queries naturally efficient, while a poorly designed one can create bottlenecks that even the best indexing can't fully overcome.

****Key Principles:****

- * ****Normalization:**** Generally, aim for a normalized schema (e.g., 3NF). This reduces data redundancy and improves data integrity, which often leads to more efficient queries because tables are smaller and focused. However, excessive normalization can lead to complex joins.
- * ****Denormalization (Judiciously):**** In some cases, for read-heavy workloads where performance is critical, you might intentionally denormalize by duplicating some data to reduce the need for joins. This is a trade-off that should be carefully considered and benchmarked.
- * ****Data Types:**** Use appropriate data types for your columns. For example, use `INT` for integers, `VARCHAR` with a

reasonable length for strings, and `DATE`/`TIMESTAMP` for dates. Using overly large types (e.g., `VARCHAR(MAX)` when `VARCHAR(50)` suffices) can increase storage and processing overhead.

- * **Primary Keys and Foreign Keys:** Properly define primary keys (which are automatically indexed) and foreign keys. Foreign keys enforce referential integrity and can be used by the optimizer to understand relationships between tables.
- * **Table Partitioning:** For very large tables, consider partitioning them based on a relevant key (e.g., by date). This can significantly improve query performance by allowing the database to scan only relevant partitions.

Example:

Imagine an e-commerce system.

Poor Design: Storing the product name directly in the `order\_items` table.

```
...
orders
  order_id (PK)
  customer_id
  order_date

order_items
  order_item_id (PK)
  order_id
  product_name <-- Redundant and prone to errors
  quantity
  price

products
  product_id (PK)
  product_name <-- Actual product name
  description
...
```

If the product name changes, you'd have to update it in potentially many `order\_items` records, which is inefficient and error-prone.

Better Design (Normalized):

```
...
orders
  order_id (PK)
  customer_id
  order_date

order_items
  order_item_id (PK)
  order_id
  product_id <-- Foreign Key to products
  quantity
  price

products
  product_id (PK)
  product_name
  description
...
```

Now, to get the product name for an order item, you join `order\_items` with `products` on `product\_id`. This is cleaner, more efficient for updates, and allows you to index `product\_id` in `order\_items` for faster joins.

4. Execution Plan Analysis

Explanation: This is arguably the most critical step. The database doesn't just blindly execute your SQL. It has a sophisticated query optimizer that analyzes your query and determines the most efficient way to retrieve the data. The "execution plan" (or "query plan") is a representation of this chosen strategy. Understanding it is like looking under the hood to see exactly what the database is doing.

How to Analyze:

- * **`EXPLAIN` (or `EXPLAIN PLAN`, `EXPLAIN ANALYZE`):** Most database systems provide a command to show the execution plan.
 - * **`EXPLAIN`:** Shows the *estimated* plan.
 - * **`EXPLAIN ANALYZE`** (PostgreSQL, some others): Executes the query and shows the *actual* plan with timings and row counts. This is invaluable for identifying discrepancies between estimates and reality.
- * **Key Things to Look For in an Execution Plan:**
 - * **Full Table Scans:** If you see "Seq Scan" (Sequential Scan) or "Table Scan" on large tables where you expect an index to be used, that's a red flag.
 - * **Index Usage:** Does the plan show "Index Scan" or "Index Only Scan"? Which index is being used?
 - * **Join Methods:** Understand the different join methods (e.g., Nested Loop, Hash Join, Merge Join) and why the optimizer chose a particular one.

* ****Row Estimates vs. Actual Rows:**** If the estimated number of rows is vastly different from the actual number of rows processed, it can indicate stale statistics or a poorly performing index.

* ****Cost:**** The plan usually shows a "cost" associated with each operation. While not an absolute measure, higher costs generally indicate more expensive operations.

* ****Sorting and Grouping:**** How is the database performing `ORDER BY` or `GROUP BY` operations? Are they using indexes, or doing costly disk sorts?

****Example (Conceptual - syntax varies by RDBMS):****

Let's assume we have the `users` table and the query from the indexing section:

```
```sql
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'john.doe@example.com';
```
```

****Possible Output (with and without index):****

****Without Index:****

```
```
 QUERY PLAN

Seq Scan on users (cost=0.00..150000.00 rows=100000 width=100) (actual time=0.050..1500.500 rows=1 *
Filter: (email = 'john.doe@example.com')
Rows Removed by Filter: 199999
Planning Time: 0.100 ms
Execution Time: 1500.600 ms
```
```

* ****Problem:**** "Seq Scan on users" indicates a full table scan. High `Execution Time`.

****With Index (`idx\_users\_email`):****

```
```
 QUERY PLAN

Index Scan using idx_users_email on users (cost=0.29..8.31 rows=1 width=100) (actual time=0.020..0.021 rows=1 *
Index Cond: (email = 'john.doe@example.com')
Planning Time: 0.050 ms
Execution Time: 0.030 ms
```
```

* ****Improvement:**** "Index Scan using idx_users_email" shows the index is used. Significantly lower `Execution Time`.

****Conclusion:****

Optimizing SQL queries is an iterative process. Start by understanding your data and how it's accessed. Implement strong indexing, write clear and efficient queries, design a robust schema, and most importantly, use `EXPLAIN ANALYZE` to understand and verify your optimizations. Don't be afraid to experiment and measure the impact of your changes.

Model: gemini-2.5-flash-lite

Token usage (estimated):

Input tokens: 97
Output tokens: 3062
Total tokens: 3159

USER PROMPT:

You are a senior backend engineer. A junior developer asked you how to optimize SQL queries for better performance. Follow the Skeleton of Thought approach:

Step 1: Generate only the skeleton of your answer in 3-5 concise bullet points.
Step 2: Expand each bullet point into a clear and detailed explanation with examples.
Make sure the final answer is structured and easy to follow.

LLM RESPONSE:

Hey there! That's a fantastic question, and one of the most crucial skills you'll develop as a backend engineer. Optimizing SQL queries is key to building performant and scalable applications. Let's break down how you can approach it.

Here's the skeleton of our discussion:

- * ****Leverage Indexes Wisely****
- * ****Craft Efficient Query Statements****
- * ****Understand the Query Execution Plan****
- * ****Optimize Database Schema Design****

Now, let's dive into each point with more detail and examples.

1. Leverage Indexes Wisely

****What it is:**** Think of an index like the index at the back of a textbook. Instead of reading the entire book to find a specific topic, you go to the index, find the page number, and jump directly to it. In a database, an index is a special lookup table that the database search engine can use to speed up data retrieval.

****How it helps:****

- * ****Faster `WHERE` clauses:**** If you frequently filter data using a specific column (e.g., `WHERE user\_id = 123`), an index on `user\_id` will dramatically speed up the lookup.
- * ****Faster `ORDER BY` and `GROUP BY`:**** Indexes can help the database sort and group data more efficiently.
- * ****Faster `JOIN` operations:**** When you join tables on indexed columns (especially foreign keys), the database can find matching rows much quicker.

****When to use them:****

- * ****Columns frequently used in `WHERE` clauses.****
- * ****Foreign key columns.****
- * ****Columns used in `ORDER BY` or `GROUP BY` clauses.****
- * ****Columns with high cardinality**** (many unique values, e.g., email addresses, user IDs).

****When to be cautious:****

- * ****Over-indexing:**** While indexes speed up reads, they slow down writes (INSERT, UPDATE, DELETE) because the index itself also needs to be updated. Each index also consumes disk space.
- * ****Low cardinality columns:**** Indexing a column with very few unique values (e.g., a boolean `is\_active` column) might not provide much benefit, as the database might still prefer a full table scan.

****Example:****

If you often search for users by their email:

```
```sql
```

```
-- Before indexing:
```

```
SELECT * FROM users WHERE email = 'john.doe@example.com';
```

```
-- This might be slow if the users table is large.
```

```
-- Create an index on the email column:
```

```
CREATE INDEX idx_users_email ON users (email);
```

```
-- After indexing:
```

```
SELECT * FROM users WHERE email = 'john.doe@example.com';
```

```
-- This query will now be significantly faster.
```

```
```
```

2. Craft Efficient Query Statements

The way you write your queries has a massive impact on performance. Small changes can lead to big gains.

* ****`SELECT` Only What You Need:****

- * ****Avoid `SELECT \*`:**** This pulls all columns, including potentially large text fields or binary data, which consumes more network bandwidth and memory.

- * ****Be explicit:**** List only the columns required by your application.

```
```sql
```

```
-- Bad:
```

```
SELECT * FROM products WHERE category_id = 5;
```

```
-- Good:
```

```
SELECT product_id, name, price FROM products WHERE category_id = 5;
```

```
```
```

* ****Optimize `WHERE` Clauses:****

- * ****Be specific:**** The more precise your filters, the fewer rows the database has to process.

- * ****Avoid functions on indexed columns:**** Applying a function to an indexed column in your `WHERE` clause can prevent the database from using the index.

```
```sql
```

```
-- Bad (prevents index use on order_date):
```

```
SELECT * FROM orders WHERE YEAR(order_date) = 2023;
```

```
-- Good (allows index use):
```

```
SELECT * FROM orders WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01';
```

```
```
```

- * ****Use `LIKE 'prefix%'` over `'%suffix'`:**** A `LIKE` clause with a wildcard at the beginning (`%keyword`) cannot use a standard index efficiently, as the database doesn't know where the string starts.

```
```sql
```

```
-- Bad (cannot use index efficiently):
```

```
SELECT * FROM users WHERE email LIKE '%@example.com';
```

```
-- Good (can use index if email is indexed):
```

```
SELECT * FROM users WHERE email LIKE 'john.%';
```

```
```
```

```

* **Efficient `JOIN` Operations:**
*   **Join on indexed columns:** Ensure the columns you're joining on (especially foreign keys) are indexed.
*   **Choose the right `JOIN` type:** Understand `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, etc., and use the one
that retrieves exactly the data you need. An `INNER JOIN` might be faster than a `LEFT JOIN` if you only need matching
rows.
*   **Join order (sometimes):** For some databases and complex queries, the order in which tables are joined can
influence performance. The query optimizer usually handles this, but it's something to be aware of.

* **Limit Your Results (`LIMIT` and `OFFSET`):**
*   For pagination or just getting a sample, always use `LIMIT` to restrict the number of rows returned. This
drastically reduces the data transferred and processed.
  ```sql
 -- Bad (pulls all products, then application filters):
 SELECT * FROM products ORDER BY price DESC;

 -- Good (pulls only the top 10 most expensive products):
 SELECT product_id, name, price FROM products ORDER BY price DESC LIMIT 10;

 -- Good for pagination (page 2, 10 items per page):
 SELECT product_id, name, price FROM products ORDER BY product_id LIMIT 10 OFFSET 10;
  ```

```

3. Understand the Query Execution Plan

****What it is:**** This is arguably the most powerful tool for SQL optimization. The execution plan is essentially the database's "recipe" or step-by-step strategy for executing your query. It shows you exactly how the database intends to retrieve and process the data.

****How to get it:**** Most databases provide a command like `EXPLAIN` or `EXPLAIN ANALYZE` (PostgreSQL) or `EXPLAIN PLAN` (Oracle) to show you this plan.

****What to look for:****

```

* **Table Scans (Full Table Scans):** If you see a "table scan" on a large table, it means the database is reading
every single row to find what it needs. This is often a sign that an index is missing or not being used.
* **Index Scans:** This is generally good! It means the database is efficiently using an index.
* **Join Methods:** Different join algorithms (e.g., nested loop, hash join, merge join) have different performance
characteristics.
* **Costs:** The plan will often show a "cost" associated with each operation. Higher costs indicate more expensive
operations.
* **Rows Examined/Returned:** Pay attention to how many rows the database *examined* versus how many it *returned*.
A large difference can indicate inefficient filtering.

```

****Example (PostgreSQL):****

```

```sql
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 123 AND order_date > '2023-01-01';
```

```

The output will be detailed, showing things like `Seq Scan` (full table scan) vs. `Index Scan`, `cost`, `rows`, `width`, `actual time`, etc. Learning to read these plans is a critical skill. If you see a `Seq Scan` on `orders` for `customer\_id`, it tells you to consider adding an index on `customer\_id` (or a composite index on `customer\_id`, `order\_date`).

4. Optimize Database Schema Design

Good database design from the start can prevent many performance issues down the line.

****Choose Appropriate Data Types:****

```

*   Use the smallest data type that can hold your data. For example, use `INT` instead of `BIGINT` if an integer
column will never exceed 2 billion. Use `VARCHAR(50)` instead of `VARCHAR(255)` if the maximum length is known to be
50 characters.
*   Using appropriate types saves storage space, which means more data can fit into memory, leading to faster
access.

```

****Normalization vs. Denormalization:****

```

* **Normalization:** Reduces data redundancy and improves data integrity by structuring tables to eliminate
duplicate information. Generally good for write-heavy systems.
* **Denormalization:** Intentionally introduces redundancy (e.g., storing a user's name directly in an `orders`
table instead of just `user_id`) to improve read performance for specific, common queries. This is often done in data
warehousing or for highly optimized read paths, but comes with the trade-off of increased complexity for data
consistency during writes.
*   For most transactional applications, start with a normalized design and only denormalize if specific read
queries become a bottleneck after profiling.

```

****Avoid Excessive `NULL` Values:****

```

*   `NULL` values can sometimes complicate indexing and query logic, making queries less efficient. Design your
schema to avoid `NULL`s where possible, perhaps by splitting tables or providing default values.

```

****Ensure Foreign Keys are Indexed:****

```

*   As mentioned in the indexing section, foreign key columns are almost always involved in `JOIN` operations.
Ensuring they have indexes is crucial for efficient joins.

```

****Example:****

Instead of storing a product's status as a `VARCHAR(255)` (e.g., 'In Stock', 'Out of Stock', 'Discontinued'), which takes more space and is slower to compare, you could use:

- * An `ENUM` type (if your database supports it and the values are fixed).
- * A `TINYINT` column (`0` for In Stock, `1` for Out of Stock, `2` for Discontinued) with a separate lookup table for the actual status names. This is much more efficient for storage and comparison.

Optimizing SQL is an ongoing process. Start by writing clear, correct queries, then use `EXPLAIN` to identify bottlenecks, and apply indexes and refine your queries based on what you learn. It's a journey of continuous learning and profiling! Good luck!

Model: gemini-2.5-flash

Token usage (estimated):

Input tokens: 97
Output tokens: 2354
Total tokens: 2451

USER PROMPT:

You are a senior backend engineer. A junior developer asked you how to optimize SQL queries for better performance. Follow the Skeleton of Thought approach:

Step 1: Generate only the skeleton of your answer in 3–5 concise bullet points.
Step 2: Expand each bullet point into a clear and detailed explanation with examples.
Make sure the final answer is structured and easy to follow.

LLM RESPONSE:

Hello! As a senior engineer, I'm glad you're asking this early. Writing queries that work is step one; writing queries that scale is what separates a junior from a mid-level developer.

Here is a structured approach to SQL optimization using the Skeleton of Thought method.

Step 1: The Skeleton

- * ****Effective Indexing Strategies:**** Understanding how to use B-Trees, composite indexes, and avoiding over-indexing.
- * ****Writing SARGable Queries:**** Ensuring the database can actually use your indexes by avoiding functions on columns in `WHERE` clauses.
- * ****Analyzing Execution Plans (`EXPLAIN`):**** Learning how to let the database engine tell you exactly where the bottleneck is.
- * ****Reducing Data I/O:**** Minimizing the amount of data transferred and processed by being specific with `SELECT` and `JOIN`.

Step 2: Detailed Expansion

1. Effective Indexing Strategies

Think of an index like the index at the back of a book. Without it, the database has to do a "Full Table Scan" (reading every single page).

- * ****The Basics:**** Always index columns used frequently in `WHERE`, `JOIN`, and `ORDER BY` clauses.
- * ****Composite Indexes:**** If you often query by `last\_name` AND `first\_name`, a single index on `(last\_name, first\_name)` is faster than two separate indexes. Order matters here—the most selective column should come first.
- * ****The Trade-off:**** Every index slows down `INSERT`, `UPDATE`, and `DELETE` operations because the index must be updated too. Don't index every column "just in case."

2. Writing SARGable Queries

SARGable stands for ****S**earch ****A**rgument ****A**ble****. A query is SARGable if the engine can use an index to speed it up.**

- * ****The Pitfall:**** Avoid wrapping columns in functions.
 - * ***Bad:**** `SELECT \* FROM users WHERE YEAR(created\_at) = 2023;` (This forces a full scan because the engine has to calculate the year for every row).
 - * ***Good:**** `SELECT \* FROM users WHERE created\_at >= '2023-01-01' AND created\_at <= '2023-12-31';` (This allows an index scan on `created\_at`).
- * ****Wildcards:**** `LIKE 'abc%'` can use an index. `LIKE '%abc'` cannot, because the index is sorted from the start of the string.

3. Analyzing Execution Plans (`EXPLAIN`)

Before you change code, you must measure. Every major SQL dialect (PostgreSQL, MySQL, SQL Server) has an `EXPLAIN` command.

- * ****How to use it:**** Prepend `EXPLAIN` or `EXPLAIN ANALYZE` to your query.
- * ****What to look for:****
 - * ****Seq Scan / Full Table Scan:**** This is a red flag for large tables. It means the engine is reading everything.
 - * ****Index Scan:**** This is what you want—the engine found the data via the index.
 - * ****Cost/Rows:**** Look for high numbers here; they indicate where the engine is working the hardest.

4. Reducing Data I/O

The most expensive part of a query is often moving data from the disk to the memory or over the network.

- * **Stop using `SELECT \*`:** Only fetch the columns you need. Fetching a `TEXT` or `BLOB` column you don't use wastes massive amounts of RAM and I/O.
- * **Limit your results:** If you only need to check if a record exists, use `LIMIT 1`. If you are displaying a list, use Pagination (`LIMIT` and `OFFSET`).
- * **Join Wisely:** Ensure joined columns are of the same data type. If you join an `INT` column to a `VARCHAR` column, the database may have to perform a type conversion on every single row, killing performance.

Pro-Tip: If you find yourself writing a query with deep nested subqueries or massive loops in your backend code, stop and ask: "Can I do this in one efficient JOIN or a Common Table Expression (CTE)?" Let the database do the heavy lifting!

Model: gemini-3-flash-preview

Token usage (estimated):

Input tokens: 97
Output tokens: 915
Total tokens: 1012

=====

USER PROMPT:

You are a software architect. I want you to produce an Architecture Decision Record (ADR) about choosing PostgreSQL instead of MongoDB.

Follow the Skeleton of Thought approach:

Step 1: First, output only the skeleton of the ADR as section headers (no explanations yet).

Use the standard ADR structure with 5 sections: Context, Decision, Alternatives Considered, Consequences, References.

Step 2: After showing the skeleton, expand each section with clear and detailed content.

Keep the final ADR professional, structured, and easy to read.

LLM RESPONSE:

Here's the ADR following the Skeleton of Thought approach:

ADR-00X: Choosing PostgreSQL over MongoDB for Data Storage

- ### 1. Context
- ### 2. Decision
- ### 3. Alternatives Considered
- ### 4. Consequences
- ### 5. References

ADR-00X: Choosing PostgreSQL over MongoDB for Data Storage

1. Context

The application currently requires a robust and scalable data storage solution. Key requirements include:

- * **Data Integrity and Consistency:** Ensuring that data is accurate, reliable, and adheres to defined business rules is paramount. Transactions need to be ACID compliant.
- * **Structured Data and Complex Queries:** The application will deal with highly structured data, and the ability to perform complex analytical queries, joins, and aggregations is essential for reporting and business intelligence.
- * **Scalability:** The system needs to handle increasing volumes of data and user traffic.
- * **Maturity and Ecosystem:** A mature technology with a large, active community, extensive tooling, and readily available expertise is preferred for long-term maintainability and development velocity.
- * **Schema Flexibility (Considered):** While some parts of the application might benefit from schema flexibility, the core data model is expected to be well-defined and relatively stable.

The choice of database technology will significantly impact the application's performance, reliability, development effort, and operational costs. This ADR documents the decision to select PostgreSQL as the primary data storage solution, moving away from an initial consideration of MongoDB.

2. Decision

We will use **PostgreSQL** as the primary relational database management system for our application's data storage needs.

This decision is based on PostgreSQL's strengths in data integrity, ACID compliance, support for complex queries and joins, its mature ecosystem, and its proven scalability for structured data workloads.

3. Alternatives Considered

- * **MongoDB:**
 - * **Description:** A popular NoSQL document database that stores data in flexible, JSON-like documents.
 - * **Pros:** Schema flexibility, horizontal scalability (sharding), good for rapidly evolving data structures, ease of development for certain use cases.
 - * **Cons:** Weaker guarantees for ACID transactions across multiple documents, can be more challenging for complex relational queries and joins, data consistency can be a concern without careful application-level management,


```
potential for data duplication and schema drift if not managed meticulously.
*   **Reason for not choosing:** The strong emphasis on data integrity, ACID compliance, and the need for complex relational queries and joins made MongoDB a less suitable primary choice for the core application data. While its schema flexibility is attractive in some contexts, the structured nature of our primary data and the need for strong consistency tipped the scales.

*   **MySQL:**
    *   **Description:** Another widely used open-source relational database management system.
    *   **Pros:** Mature, widely adopted, good performance, strong community support, ACID compliant.
    *   **Cons:** Historically, PostgreSQL has been considered to have a more advanced feature set, particularly in areas like advanced indexing, extensibility, and richer data types. While MySQL has made significant strides, PostgreSQL still holds an edge in certain complex analytical and data warehousing scenarios.
    *   **Reason for not choosing:** While a strong contender, PostgreSQL's richer feature set, particularly its extensibility and advanced data types, made it a slightly more compelling choice for our specific technical requirements and anticipated future needs. The decision between PostgreSQL and MySQL often comes down to nuanced feature preferences and specific workload characteristics.
```

4. Consequences

```
**Positive Consequences:**

*   **Enhanced Data Integrity and Consistency:** PostgreSQL's strong ACID compliance and robust transaction management will ensure data accuracy and reliability, reducing the risk of data corruption or inconsistencies.
*   **Simplified Complex Querying:** The ability to perform complex joins, aggregations, and analytical queries will be straightforward, supporting rich reporting and business intelligence features.
*   **Mature and Robust Ecosystem:** Access to a vast array of tools, libraries, experienced developers, and established best practices will accelerate development and simplify operations.
*   **Strong Relational Data Modeling:** The relational model is well-suited for our structured data, leading to a clear and maintainable data schema.
*   **Proven Scalability (Vertical and Horizontal):** PostgreSQL can scale effectively both vertically (more powerful hardware) and horizontally (replication, sharding with extensions like Citus) for growing workloads.
*   **Extensibility:** PostgreSQL's extensibility allows for custom data types, functions, and extensions, providing flexibility for future needs.

**Negative Consequences:**

*   **Less Schema Flexibility (compared to MongoDB):** Changes to the data schema will require more deliberate planning and migration strategies. This is a trade-off for stronger data integrity.
*   **Steeper Learning Curve for Developers New to SQL:** Developers unfamiliar with SQL and relational database concepts may require additional training.
*   **Potential for Higher Operational Overhead (initially):** Managing a relational database, especially with advanced features, might require specialized expertise and careful tuning for optimal performance.
```

5. References

```
*   [PostgreSQL Official Website](https://www.postgresql.org/)
*   [MongoDB Official Website](https://www.mongodb.com/)
*   [MySQL Official Website](https://www.mysql.com/)
*   [ACID Transaction Properties](https://en.wikipedia.org/wiki/ACID)
*   [PostgreSQL vs. MongoDB: A Comprehensive Comparison](https://www.example.com/postgre-vs-mongo-comparison)
(Hypothetical reference to a detailed comparison document)
```

```
Model: gemini-2.5-flash-lite
Token usage (estimated):
  Input tokens: 136
  Output tokens: 1518
  Total tokens: 1654
```

USER PROMPT:

```
You are a software architect. I want you to produce an Architecture Decision Record (ADR) about choosing PostgreSQL instead of MongoDB.

Follow the Skeleton of Thought approach:
Step 1: First, output only the skeleton of the ADR as section headers (no explanations yet).
Use the standard ADR structure with 5 sections: Context, Decision, Alternatives Considered, Consequences, References.
Step 2: After showing the skeleton, expand each section with clear and detailed content.
Keep the final ADR professional, structured, and easy to read.
```

LLM RESPONSE:

```
**Step 1: Skeleton of the ADR**

# Architecture Decision Record: Choosing PostgreSQL as Primary Data Store

## Context

## Decision

## Alternatives Considered
```

Consequences

References

Step 2: Expanded Content

Architecture Decision Record: Choosing PostgreSQL as Primary Data Store

Context

This ADR addresses the selection of a primary data store for the new `[Project Name/Platform Name]` application. The application is a critical platform designed to manage `[briefly describe core functionality, e.g., user profiles, orders, inventory, financial transactions]`. Key requirements for the data store include:

1. ****Data Integrity & Consistency:**** Strong guarantees for data correctness, especially for transactional operations.
2. ****Transactional Support (ACID):**** Full ACID compliance for complex business processes involving multiple data changes.
3. ****Complex Querying & Reporting:**** Ability to perform sophisticated queries, aggregations, and generate various reports efficiently.
4. ****Data Relationships:**** Support for managing complex relationships between different entities.
5. ****Scalability:**** Ability to scale with anticipated user growth and data volume.
6. ****Developer Familiarity & Ecosystem:**** Leverage existing team skills and a mature ecosystem for tooling, monitoring, and support.
7. ****Future Extensibility:**** Flexibility to adapt to evolving data models and integrate new features without major re-architecture.

Initial considerations focused on two primary database paradigms: relational (SQL) and document-oriented (NoSQL), with PostgreSQL and MongoDB emerging as leading candidates within their respective categories due to their popularity, feature sets, and community support.

Decision

We have decided to adopt ****PostgreSQL**** as the primary data store for the `[Project Name/Platform Name]` application.

The rationale for this decision is based on PostgreSQL's robust capabilities that directly address our core requirements:

- * ****Strong Data Integrity and ACID Compliance:**** PostgreSQL guarantees atomicity, consistency, isolation, and durability, which are paramount for the transactional nature of our application's core business logic. This ensures data correctness and reliability, reducing the risk of data corruption or inconsistencies.
- * ****Mature Relational Model:**** Its relational nature is well-suited for our inherently structured data and the complex relationships between entities (e.g., users, orders, products). This facilitates data modeling, querying, and maintaining referential integrity.
- * ****Powerful Querying and Reporting:**** PostgreSQL's advanced SQL capabilities, indexing options, and robust aggregation functions are ideal for the anticipated complex queries, analytical needs, and reporting requirements of the platform.
- * ****Flexibility with JSONB:**** While a relational database, PostgreSQL's native JSONB support provides excellent flexibility to store and query semi-structured or schema-less data where appropriate, effectively bridging the gap between relational and document databases for specific use cases.
- * ****Mature Ecosystem and Community:**** PostgreSQL benefits from a vast and active community, extensive tooling, and a proven track record in enterprise environments, offering strong support, stability, and a rich feature set.

Alternatives Considered

1. MongoDB

- * ****Description:**** MongoDB is a popular NoSQL, document-oriented database that stores data in flexible, JSON-like documents. It is designed for horizontal scalability and high performance with large volumes of data.
- * ****Pros:****
 - * ****Schema Flexibility:**** Documents do not require a predefined schema, allowing for rapid iteration and accommodating evolving data models easily, especially for rapidly changing data structures.
 - * ****Scalability:**** Excellent horizontal scaling capabilities (sharding) out-of-the-box, making it suitable for very large, distributed datasets.
 - * ****Ease of Development:**** Often perceived as easier for developers working with object-oriented languages due to its document model aligning well with application objects.
 - * ****Performance:**** Can offer high write performance for specific workloads due to its append-only writes and eventual consistency model.
- * ****Cons:****
 - * ****Weaker Transactional Guarantees:**** While MongoDB introduced multi-document ACID transactions in recent versions, their implementation and performance characteristics can be more complex and less mature than traditional relational databases. By default, it operates with eventual consistency.
 - * ****Data Integrity Challenges:**** Lack of strict schema enforcement can lead to data inconsistencies if not carefully managed at the application layer. Referential integrity (joins) is typically handled by the application, making complex relationships harder to manage and query across.
 - * ****Complex Aggregations and Reporting:**** While MongoDB has an aggregation framework, performing complex joins, analytical queries, and sophisticated reporting across multiple "collections" can be more cumbersome and less performant compared to SQL.

* ****Learning Curve for Operations:**** Managing and optimizing a sharded MongoDB cluster can introduce operational complexity.

* ****Reason for not choosing:**** The primary reasons for not choosing MongoDB were the paramount importance of strong ACID transactional guarantees, strict data integrity, and the anticipated need for complex, multi-entity queries and reporting. While MongoDB has evolved, its core design principles prioritize flexibility and horizontal scalability over the strong consistency and relational integrity that are critical for `[Project Name/Platform Name]`. The overhead of enforcing integrity and managing complex relationships at the application level was deemed too high a risk.

2. Other Relational Databases (e.g., MySQL)

* ****Description:**** MySQL is another widely used open-source relational database.

* ****Reason for not choosing:**** While a viable option, PostgreSQL was preferred over MySQL due to its more advanced feature set, especially regarding:

* ****Advanced SQL Features:**** PostgreSQL generally offers richer and more standard-compliant SQL features, including advanced window functions, common table expressions (CTEs), and more sophisticated indexing options.

* ****JSONB Support:**** PostgreSQL's native JSONB type and associated functions are superior for handling semi-structured data compared to MySQL's JSON capabilities.

* ****Extensibility:**** PostgreSQL's extensibility (e.g., custom data types, operators, procedural languages) provides greater flexibility for future specialized needs.

* ****Licensing:**** PostgreSQL's permissive PostgreSQL License is often preferred over MySQL's dual-licensing model (GPL/Commercial).

Consequences

Positive Consequences:

* ****Guaranteed Data Integrity and Consistency:**** Ensures the reliability and correctness of business-critical data through ACID transactions and robust schema enforcement.

* ****Robust Transactional Support:**** Simplifies the implementation of complex business logic requiring multi-statement atomicity.

* ****Powerful Querying and Analytics:**** Enables efficient execution of complex queries, aggregations, and supports advanced reporting and business intelligence needs without requiring separate analytical stores.

* ****Schema Evolution Control:**** Provides structured schema evolution through migrations, ensuring data consistency and manageability over time.

* ****Developer Productivity:**** Leveraging a mature ORM with PostgreSQL allows developers to focus on business logic while benefiting from database integrity. Existing team members with SQL experience can quickly become productive.

* ****Future-Proofing:**** PostgreSQL's extensibility (e.g., PostGIS for geo-spatial, full-text search) provides a solid foundation for future features and domain-specific requirements.

* ****Cost-Effective:**** As a powerful open-source solution, it avoids proprietary licensing costs while offering enterprise-grade features.

Negative Consequences / Trade-offs:

* ****Initial Schema Design Effort:**** Requires upfront design and planning for the relational schema, which can be more time-consuming than a schema-less approach.

* ****Vertical Scaling Preference:**** While sharding is possible, PostgreSQL is often scaled vertically initially. Horizontal scaling (e.g., read replicas, sharding) requires more explicit planning and architectural considerations compared to some NoSQL databases designed for distributed scale from the ground up.

* ****SQL/ORM Learning Curve:**** Developers new to SQL or ORMs will need to become proficient.

* ****Less Flexible for Truly Unstructured Data:**** While JSONB helps, deeply nested, highly variable, and truly unstructured data might still be more naturally modeled in a pure document database.

Mitigation Strategies:

* ****Standardize ORM:**** Adopt a well-supported Object-Relational Mapper (ORM) (e.g., `[Specific ORM, e.g., TypeORM for TypeScript, SQLAlchemy for Python, Hibernate for Java]`) to streamline database interactions and abstract SQL complexities for developers.

* ****Database Migration Strategy:**** Implement a robust database migration tool (e.g., `[Specific Tool, e.g., Flyway, Liquibase, TypeORM Migrations]`) to manage schema changes in a controlled and versioned manner.

* ****Performance Monitoring:**** Implement comprehensive database performance monitoring and logging to proactively identify and address bottlenecks.

* ****Scaling Strategy:**** Plan for scaling strategies, including read replicas for read-heavy workloads and exploring logical sharding for specific high-volume data partitions if necessary in the future.

* ****Developer Training:**** Provide training and resources for developers on PostgreSQL best practices, SQL, and the chosen ORM.

References

* PostgreSQL Official Documentation: <https://www.postgresql.org/docs/>

* MongoDB Official Documentation: <https://www.mongodb.com/docs/>

* Internal Discussion on Database Selection (Link to Confluence/Jira/Slack thread if applicable)

* [Optional: Link to a specific benchmark or comparison article that influenced the decision]

Model: gemini-2.5-flash

Token usage (estimated):

Input tokens: 136

Output tokens: 2583

Total tokens: 2719

USER PROMPT:

You are a software architect. I want you to produce an Architecture Decision Record (ADR) about choosing PostgreSQL instead of MongoDB.

Follow the Skeleton of Thought approach:

Step 1: First, output only the skeleton of the ADR as section headers (no explanations yet).

Use the standard ADR structure with 5 sections: Context, Decision, Alternatives Considered, Consequences, References.

Step 2: After showing the skeleton, expand each section with clear and detailed content.

Keep the final ADR professional, structured, and easy to read.

LLM RESPONSE:

As a Software Architect, I have prepared the following Architecture Decision Record (ADR) regarding the selection of our primary database engine.

Step 1: Skeleton of the ADR

1. **Context**
2. **Decision**
3. **Alternatives Considered**
4. **Consequences**
5. **References**

Step 2: Full ADR Expansion

ADR 004: Selection of PostgreSQL as Primary Database System

Status: Decided
Date: 2023-10-27
Decider: Software Architect

1. Context

Our application requires a robust data storage solution to handle core business entities including users, transactions, and relational configurations. The following technical requirements and constraints were identified:

- * **Data Integrity:** We require strict ACID (Atomicity, Consistency, Isolation, Durability) compliance to ensure financial and transactional data remains accurate.
- * **Relational Complexity:** The domain model involves deeply nested relationships that frequently require complex joins and aggregations.
- * **Schema Evolution:** While we need agility, we also need to enforce a contract between the data layer and the application to prevent "schema drift."
- * **Extensibility:** A need for both structured relational data and occasionally semi-structured metadata (e.g., external API responses).
- * **Team Expertise:** The engineering team has high proficiency in SQL and relational modeling.

2. Decision

We will use **PostgreSQL** as our primary relational database management system (RDBMS).

PostgreSQL was chosen because it provides the best balance between traditional relational rigidity and modern flexibility. Specifically:

- * **JSONB Support:** PostgreSQL allows us to store and index semi-structured data, effectively covering many use cases where a document store like MongoDB would typically be used.
- * **Advanced Indexing:** It offers superior indexing options (GIN, GiST, BRIN) which are critical for our performance requirements.
- * **Reliability:** It has a long-standing reputation for data correctness and handles high-concurrency workloads effectively.

3. Alternatives Considered

MongoDB (NoSQL Document Store)

- * **Pros:** High write throughput, horizontal scaling (sharding) out of the box, and a flexible schema that allows for rapid iteration without migrations.
- * **Cons:** Managing relationships (joins) at the application level leads to increased complexity and latency. Maintaining data consistency across documents requires "Two-Phase Commits" in code, which is error-prone. The lack of a forced schema can lead to data quality issues over time in a multi-service environment.

MySQL

- * **Pros:** Widely available, excellent performance for read-heavy workloads.
- * **Cons:** Historically less feature-rich than PostgreSQL regarding complex analytical queries and custom data types (though the gap is closing, PostgreSQL remains the standard for feature-richness).

4. Consequences

Positive:

- * **Strong Consistency:** We can rely on the database to enforce data integrity through foreign keys and constraints.
- * **Unified Tooling:** By using JSONB within PostgreSQL, we avoid the operational overhead of managing two different database technologies (Polyglot Persistence) for relational and document needs.

* ****Maturity:**** Access to a vast ecosystem of extensions (e.g., PostGIS for geography, TimescaleDB for time-series).

****Negative/Risks:****

* ****Horizontal Scaling:**** While PostgreSQL scales vertically very well, horizontal scaling (sharding) is more complex to implement compared to MongoDB.

* ****Migrations:**** Changes to the schema will require structured migration scripts and potential downtime or "expand and contract" patterns for large tables.

5. References

* [PostgreSQL Official Documentation](https://www.postgresql.org/docs/)

* [PostgreSQL JSON Types (JSONB vs JSON)](https://www.postgresql.org/docs/current/datatype-json.html)

* [ACID vs. BASE consistency models](https://www.sciencedirect.com/topics/computer-science/base-consistency)

* [Internal Benchmark Report: SQL Joins vs Document Embedding (v1.0.2)](https://internal-wiki.example.com/benchmarks)

Model: gemini-3-flash-preview

Token usage (estimated):

Input tokens: 136

Output tokens: 1040

Total tokens: 1176

=====
USER PROMPT:

You are a senior Go developer. I want you to help me plan a REST API for managing products in Go.

Follow the Skeleton of Thought approach:

Step 1: Output only the skeleton of the solution in 6–8 concise bullet points.

The skeleton must cover: data model definition in Go (structs), choice of HTTP framework or net/http, routing, handlers, validations, database layer, error handling, and project structure. Do not expand yet.

Step 2: Expand each bullet point with clear technical details, examples, and Go best practices.

Include sample code snippets in Go (structs, handlers, routes) and considerations about packages (e.g., chi, or net/http), error handling with idiomatic Go, and how to organize the project into packages (handlers, models, db). Use concise and professional language.

The API must implement CRUD operations for products with fields: id, name, description, price, stock.

LLM RESPONSE:

Okay, let's plan this Go REST API for product management.

Step 1: Skeleton of Thought

* ****Data Model Definition:**** Define Go structs for `Product` and related entities.
* ****HTTP Framework/Library:**** Choose between `net/http` and a microframework like `chi`.
* ****Routing:**** Implement a clear routing strategy.
* ****Handlers:**** Design handler functions for CRUD operations.
* ****Validations:**** Implement input validation for incoming requests.
* ****Database Layer:**** Abstract database interactions.
* ****Error Handling:**** Establish a consistent error handling mechanism.
* ****Project Structure:**** Organize code into logical packages.

Step 2: Expanded Technical Details

Data Model Definition

We'll define the `Product` struct to represent our data. For database interactions, we'll likely use a struct that mirrors the database schema, potentially with additional fields like `CreatedAt` and `UpdatedAt`.

```
``go
package models
```

```
import "time"
```

```
// Product represents a product in the system.
```

```
type Product struct {
    ID          int        `json:"id"`
    Name        string     `json:"name"`
    Description string     `json:"description"`
    Price       float64    `json:"price"`
    Stock       int        `json:"stock"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}
```

```
// ProductCreateRequest is used for creating new products.
```

```
// We might exclude auto-generated fields like ID, CreatedAt, UpdatedAt.
type ProductCreateRequest struct {
    Name      string `json:"name"`
    Description string `json:"description"`
    Price     float64 `json:"price"`
    Stock     int    `json:"stock"`
}

// ProductUpdateRequest is used for updating existing products.
type ProductUpdateRequest struct {
    Name      *string `json:"name,omitempty"` // Use pointers for optional updates
    Description *string `json:"description,omitempty"`
    Price     *float64 `json:"price,omitempty"`
    Stock     *int    `json:"stock,omitempty"`
}
```


Best Practices:

- Use `json` tags for clear JSON serialization/deserialization.
- Create separate request/response structs to decouple API contracts from the core data model.
- Use pointers for fields in update requests to allow partial updates.

HTTP Framework/Library

While `net/http` is Go's standard library, for a REST API with routing, middleware, and better organization, a microframework like `chi` is highly recommended. It provides a robust routing mechanism and middleware support without being overly opinionated.


```
```go
// Example using chi
package main

import (
 "net/http"

 "github.com/go-chi/chi/v5"
 "github.com/go-chi/chi/v5/middleware"
 "your_module_path/handlers" // Assuming handlers are in a 'handlers' package
)

func main() {
 r := chi.NewRouter()

 // Middleware
 r.Use(middleware.Logger) // Log HTTP requests
 r.Use(middleware.Recoverer) // Recover from panics
 r.Use(middleware.StripSlashes) // Remove trailing slashes

 // Routes
 r.Route("/products", func(r chi.Router) {
 r.Post("/", handlers.CreateProduct) // POST /products
 r.Get("/", handlers.ListProducts) // GET /products
 r.Get("/{id}", handlers.GetProduct) // GET /products/{id}
 r.Put("/{id}", handlers.UpdateProduct) // PUT /products/{id}
 r.Delete("/{id}", handlers.DeleteProduct) // DELETE /products/{id}
 })

 // Start server
 port := ":8080"
 http.ListenAndServe(port, r)
}
```
```


Considerations:

- net/http: Good for simple APIs or when you want minimal dependencies. Requires manual routing and middleware implementation.
- chi: Lightweight, fast, and provides excellent routing and middleware capabilities. Highly idiomatic and well-maintained. Other options include `gorilla/mux` or `gin-gonic`.

Routing

`chi`'s router provides a clear and hierarchical way to define API endpoints. We'll map HTTP methods and paths to specific handler functions.


```
```go
// (See example in the HTTP Framework section above)
```
```


Best Practices:

- Use RESTful conventions for URL design (e.g., `/products`, `/products/{id}`).

```

- \* Group related routes under a common prefix (e.g., ``/products``).
- \* Leverage path parameters for identifying specific resources (e.g., ``{id}``).

### ### Handlers

Handler functions will receive ``http.ResponseWriter`` and ``*http.Request``. They'll parse the request, interact with the database layer, and write the response.

```
```go
package handlers

import (
    "encoding/json"
    "net/http"
    "strconv"

    "github.com/go-chi/chi/v5"
    "your_module_path/models"
    "your_module_path/services" // Assuming database/business logic is in a 'services' package
)

// CreateProduct handles POST /products requests.
func CreateProduct(w http.ResponseWriter, r *http.Request) {
    var req models.ProductCreateRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request payload", http.StatusBadRequest)
        return
    }

    // Perform validation here or delegate to a service
    if req.Name == "" || req.Price <= 0 || req.Stock < 0 {
        http.Error(w, "Invalid product data", http.StatusBadRequest)
        return
    }

    product, err := services.ProductService.Create(r.Context(), req)
    if err != nil {
        // Handle specific errors from service (e.g., duplicate name)
        http.Error(w, err.Error(), http.StatusInternalServerError) // Placeholder
        return
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(product)
}

// GetProduct handles GET /products/{id} requests.
func GetProduct(w http.ResponseWriter, r *http.Request) {
    idStr := chi.URLParam(r, "id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        http.Error(w, "Invalid product ID", http.StatusBadRequest)
        return
    }

    product, err := services.ProductService.GetByID(r.Context(), id)
    if err != nil {
        // Handle not found, internal errors
        if err == services.ErrProductNotFound { // Assuming ErrProductNotFound is defined in services
            http.Error(w, "Product not found", http.StatusNotFound)
        } else {
            http.Error(w, "Internal server error", http.StatusInternalServerError)
        }
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(product)
}

// Add similar handlers for ListProducts, UpdateProduct, DeleteProduct
```
```

### \*\*Best Practices:\*\*

- \* Keep handlers lean; delegate complex logic to service or repository layers.
- \* Use ``context.Context`` for request-scoped values and cancellation signals.
- \* Set appropriate HTTP status codes.
- \* Handle request body decoding and response encoding.

### ### Validations

Input validation is crucial to ensure data integrity. This can be done within handlers, but it's often cleaner to delegate to a validation package or the service layer.

```
```go
// Example of validation logic within a handler (can be moved to a dedicated package)
func CreateProduct(w http.ResponseWriter, r *http.Request) {
    var req models.ProductCreateRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request payload", http.StatusBadRequest)
        return
    }

    // Basic validation
    if req.Name == "" {
        http.Error(w, "Name is required", http.StatusBadRequest)
        return
    }
    if req.Price <= 0 {
        http.Error(w, "Price must be positive", http.StatusBadRequest)
        return
    }
    if req.Stock < 0 {
        http.Error(w, "Stock cannot be negative", http.StatusBadRequest)
        return
    }

    // ... proceed to service layer
}

// For more complex validation, consider libraries like "github.com/go-playground/validator/v10"
// and integrate it into your models or request structs.
```
```

```
Best Practices:
* Validate required fields, data types, ranges, and formats.
* Return informative error messages with `http.StatusBadRequest`.
* Consider using a dedicated validation library for complex scenarios.
```

### ### Database Layer

Abstracting database operations is key for testability and maintainability. We'll define an interface for database operations and implement it for a specific database (e.g., PostgreSQL, MySQL, SQLite).

```
```go
package db

import (
    "context"
    "database/sql"
    "fmt"
    "time"

    "your_module_path/models"
)

// ProductRepository defines the interface for product database operations.
type ProductRepository interface {
    Create(ctx context.Context, product *models.Product) error
    GetByID(ctx context.Context, id int) (*models.Product, error)
    GetAll(ctx context.Context) ([]models.Product, error)
    Update(ctx context.Context, product *models.Product) error
    Delete(ctx context.Context, id int) error
}

// SQLProductRepository is an implementation of ProductRepository using SQL.
type SQLProductRepository struct {
    db *sql.DB
}

// NewSQLProductRepository creates a new SQLProductRepository.
func NewSQLProductRepository(db *sql.DB) *SQLProductRepository {
    return &SQLProductRepository{db: db}
}

// Create inserts a new product into the database.
func (r *SQLProductRepository) Create(ctx context.Context, product *models.Product) error {
    query := `
        INSERT INTO products (name, description, price, stock, created_at, updated_at)
    `
```



```

        VALUES ($1, $2, $3, $4, $5, $6) RETURNING id
    `
    err := r.db.QueryRowContext(ctx, query,
        product.Name, product.Description, product.Price, product.Stock, time.Now(), time.Now(),
    ).Scan(&product.ID)
    if err != nil {
        return fmt.Errorf("failed to create product: %w", err)
    }
    return nil
}

// GetByID retrieves a product by its ID.
func (r *SQLProductRepository) GetByID(ctx context.Context, id int) (*models.Product, error) {
    query := `SELECT id, name, description, price, stock, created_at, updated_at FROM products WHERE id = $1`
    var p models.Product
    err := r.db.QueryRowContext(ctx, query, id).Scan(&p.ID, &p.Name, &p.Description, &p.Price, &p.Stock,
    &p.CreatedAt, &p.UpdatedAt)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, sql.ErrNoRows // Return specific error for "not found"
        }
        return nil, fmt.Errorf("failed to get product by ID %d: %w", id, err)
    }
    return &p, nil
}

```

```

// Add implementations for GetAll, Update, Delete
```

```

```

Best Practices:
* Use an interface to decouple the application logic from the specific database implementation.
* Pass `context.Context` to all database operations.
* Handle potential database errors (e.g., `sql.ErrNoRows`).
* Consider using an ORM (like `GORM`) or a query builder for more complex applications.

```

### ### Error Handling

Idiomatic Go error handling involves returning errors and checking them. For APIs, we need to translate these errors into appropriate HTTP status codes.

```

```go
package services

import (
    "context"
    "errors"
    "fmt"

    "your_module_path/db"
    "your_module_path/models"
)

// Define common errors
var (
    ErrProductNotFound = errors.New("product not found")
    ErrInvalidInput    = errors.New("invalid input")
)

// ProductService defines business logic operations.
type ProductService interface {
    Create(ctx context.Context, req models.ProductCreateRequest) (*models.Product, error)
    GetByID(ctx context.Context, id int) (*models.Product, error)
    // Add other methods
}

// productService implements ProductService.
type productService struct {
    repo db.ProductRepository
}

// NewProductService creates a new ProductService.
func NewProductService(repo db.ProductRepository) ProductService {
    return &productService{repo: repo}
}

// Create handles product creation logic.
func (s *productService) Create(ctx context.Context, req models.ProductCreateRequest) (*models.Product, error) {
    // Perform business logic validations (can overlap with handler validations)
    if req.Name == "" {
        return nil, fmt.Errorf("%w: name cannot be empty", ErrInvalidInput)
    }
}

```

```

    }
    if req.Price <= 0 {
        return nil, fmt.Errorf("%w: price must be positive", ErrInvalidInput)
    }

    product := &models.Product{
        Name:      req.Name,
        Description: req.Description,
        Price:     req.Price,
        Stock:     req.Stock,
    }

    if err := s.repo.Create(ctx, product); err != nil {
        return nil, fmt.Errorf("service: failed to create product: %w", err)
    }
    return product, nil
}

// GetByID handles retrieving a product.
func (s *productService) GetByID(ctx context.Context, id int) (*models.Product, error) {
    product, err := s.repo.GetByID(ctx, id)
    if err != nil {
        if errors.Is(err, sql.ErrNoRows) { // Check for specific underlying error
            return nil, ErrProductNotFound
        }
        return nil, fmt.Errorf("service: failed to get product by ID %d: %w", id, err)
    }
    return product, nil
}

// In handlers, you would check for these specific errors:
// if errors.Is(err, services.ErrProductNotFound) { ... }
```

```

#### **\*\*Best Practices:\*\***

- \* Use `errors.Wrap` (from `pkg/errors` or `fmt.Errorf` with `%w`) to add context to errors.
- \* Define custom error variables for specific application errors (e.g., `ErrProductNotFound`).
- \* Use `errors.Is` to check for specific error types.
- \* Translate repository/service errors into appropriate HTTP status codes in handlers.

#### **### Project Structure**

A well-organized project is easier to navigate and maintain. A common structure for Go web applications is:

```

...
your_module_path/
├── cmd/
│ └── server/
│ └── main.go # Entry point, server setup
├── internal/ # Private application code, not exported
│ ├── config/ # Configuration loading
│ ├── db/ # Database interface and implementations
│ ├── handlers/ # HTTP request handlers
│ ├── models/ # Data structures (structs)
│ ├── services/ # Business logic, service interfaces
│ └── middleware/ # Custom HTTP middleware
├── pkg/ # Publicly usable libraries (optional)
├── api/ # OpenAPI/Swagger definitions (optional)
├── go.mod # Go module file
└── go.sum
...

```

#### **\*\*Explanation:\*\***

- \* `cmd/server/main.go`: The main application entry point. Initializes dependencies, sets up routing, and starts the HTTP server.
- \* `internal/`: Contains the core logic of your application. This is preferred over `pkg/` for application-specific code to prevent external dependencies.
  - \* `config/`: Handles loading configuration from files or environment variables.
  - \* `db/`: Defines the database interface and concrete implementations.
  - \* `handlers/`: Contains the HTTP request handlers.
  - \* `models/`: Defines data structures used throughout the application.
  - \* `services/`: Implements the business logic, often orchestrating calls to the `db` layer.
- \* `pkg/`: For reusable code that might be imported by other projects.
- \* `api/`: If you're using tools like Swagger/OpenAPI, definitions can go here.

#### **\*\*Best Practices:\*\***

- \* Separate concerns into distinct packages.
- \* Use `internal/` for application-specific code.
- \* Keep `main.go` clean, focusing on bootstrapping.
- \* Dependency injection is often managed in `main.go` or a dedicated setup function.

Model: gemini-2.5-flash-lite

**Token usage (estimated):**

Input tokens: 224  
Output tokens: 3718  
Total tokens: 3942

**USER PROMPT:**

You are a senior Go developer. I want you to help me plan a REST API for managing products in Go.

Follow the Skeleton of Thought approach:

Step 1: Output only the skeleton of the solution in 6–8 concise bullet points.

The skeleton must cover: data model definition in Go (structs), choice of HTTP framework or net/http, routing, handlers, validations, database layer, error handling, and project structure. Do not expand yet.

Step 2: Expand each bullet point with clear technical details, examples, and Go best practices.

Include sample code snippets in Go (structs, handlers, routes) and considerations about packages (e.g., chi, or net/http), error handling with idiomatic Go, and how to organize the project into packages (handlers, models, db). Use concise and professional language.

The API must implement CRUD operations for products with fields: id, name, description, price, stock.

**LLM RESPONSE:**

Here's a plan for your Go REST API for product management, following the Skeleton of Thought approach:

---

### Step 1: Skeleton of the Solution

1. **Data Model Definition:** Go structs for `Product` and request/response DTOs.
2. **HTTP Framework Choice:** Leverage `net/http` with `go-chi/chi` for routing.
3. **Routing Strategy:** Define RESTful endpoints for CRUD operations.
4. **Handler Implementation:** Functions to process HTTP requests and interact with the service layer.
5. **Input Validation:** Ensure incoming request data meets business rules.
6. **Database Layer:** An interface-based repository for product persistence.
7. **Error Handling:** Idiomatic Go error returns, custom errors, and HTTP status mapping.
8. **Project Structure:** Modular organization into logical packages.

---

### Step 2: Expanded Solution Details and Examples

#### 1. Data Model Definition

Define the core `Product` struct and separate DTOs (Data Transfer Objects) for requests to ensure clean separation between the domain model and API input/output.

```
```go
// internal/product/model.go

package product

import "time"

// Product represents the core product entity.
type Product struct {
    ID          string    `json:"id"`
    Name        string    `json:"name"`
    Description  string    `json:"description"`
    Price       float64   `json:"price"`
    Stock       int       `json:"stock"`
    CreatedAt   time.Time `json:"createdAt"`
    UpdatedAt   time.Time `json:"updatedAt"`
}

// CreateProductRequest defines the structure for creating a new product.
type CreateProductRequest struct {
    Name        string    `json:"name"`
    Description  string    `json:"description"`
    Price       float64   `json:"price"`
    Stock       int       `json:"stock"`
}

// UpdateProductRequest defines the structure for updating an existing product.
type UpdateProductRequest struct {
    Name        *string   `json:"name,omitempty"`
    Description *string   `json:"description,omitempty"`
}
```

```

    Price      *float64 `json:"price,omitempty"`
    Stock      *int      `json:"stock,omitempty"`
}
...

```

2. HTTP Framework Choice

We'll use `net/http` for the server and `go-chi/chi` as a lightweight, flexible router. `chi` is an excellent choice as it's built on `net/http`, offers robust middleware support, and simplifies routing compared to bare `net/http` multiplexers.

```

```go
// cmd/api/main.go

package main

import (
 "context"
 "log"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"

 "github.com/go-chi/chi/v5"
 "github.com/go-chi/chi/v5/middleware"
 "your_module_name/internal/product"
 "your_module_name/internal/product/repository" // Example: in-memory or postgres
)

func main() {
 // Initialize repository (e.g., in-memory for simplicity, or PostgreSQL)
 productRepo := repository.NewInMemoryProductRepository() // Or repository.NewPostgresProductRepository(...)

 // Initialize product handler with the repository
 productHandler := product.NewHandler(productRepo)

 r := chi.NewRouter()
 r.Use(middleware.Logger) // Request logger
 r.Use(middleware.Recoverer) // Catches panics and recovers
 r.Use(middleware.Timeout(60 * time.Second)) // Request timeout

 // Product routes
 r.Route("/products", func(r chi.Router) {
 r.Post("/", productHandler.CreateProduct)
 r.Get("/", productHandler.ListProducts)
 r.Get("/{id}", productHandler.GetProductByID)
 r.Put("/{id}", productHandler.UpdateProduct)
 r.Delete("/{id}", productHandler.DeleteProduct)
 })

 server := &http.Server{
 Addr: ":8080",
 Handler: r,
 }

 // Start server in a goroutine
 go func() {
 log.Printf("Server starting on %s", server.Addr)
 if err := server.ListenAndServe(); err != nil && err != http.ErrServerClosed {
 log.Fatalf("Could not listen on %s: %v\n", server.Addr, err)
 }
 }()

 // Graceful shutdown
 quit := make(chan os.Signal, 1)
 signal.Notify(quit, os.Interrupt, syscall.SIGTERM)
 <-quit

 log.Println("Server shutting down...")
 ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
 defer cancel()

 if err := server.Shutdown(ctx); err != nil {
 log.Fatalf("Server forced to shutdown: %v", err)
 }

 log.Println("Server exited gracefully.")
}
...

```

### #### 3. Routing Strategy

`chi` provides a clean and concise way to define routes, including URL parameters and route groups. We'll define standard RESTful endpoints for CRUD.

```
```go
// Part of cmd/api/main.go (as shown above)

// Product routes
r.Route("/products", func(r chi.Router) {
    r.Post("/", productHandler.CreateProduct)           // Create a new product
    r.Get("/", productHandler.ListProducts)             // List all products
    r.Get("/{id}", productHandler.GetProductByID)       // Get a product by ID
    r.Put("/{id}", productHandler.UpdateProduct)        // Update a product by ID
    r.Delete("/{id}", productHandler.DeleteProduct)     // Delete a product by ID
})
```
```

### #### 4. Handler Implementation

Handlers are `http.HandlerFunc` functions that decode requests, call the appropriate repository method, and encode responses. They should focus on HTTP concerns, delegating business logic to the repository/service layer.

```
```go
// internal/product/handler.go

package product

import (
    "encoding/json"
    "errors"
    "net/http"
    "time"

    "github.com/go-chi/chi/v5"
    "github.com/google/uuid"
)

// Repository defines the interface for product data operations.
// This allows for dependency injection and easy switching of persistence layers.
type Repository interface {
    Create(p Product) error
    GetByID(id string) (Product, error)
    Update(p Product) error
    Delete(id string) error
    List() ([]Product, error)
}

// Handler provides HTTP handlers for product management.
type Handler struct {
    repo Repository
}

// NewHandler creates a new product handler.
func NewHandler(repo Repository) *Handler {
    return &Handler{repo: repo}
}

// CreateProduct handles POST /products requests.
func (h *Handler) CreateProduct(w http.ResponseWriter, r *http.Request) {
    var req CreateProductRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request payload", http.StatusBadRequest)
        return
    }

    // Basic validation
    if req.Name == "" || req.Price <= 0 || req.Stock < 0 {
        http.Error(w, "Name, Price (must be > 0), and Stock (must be >= 0) are required fields.",
            http.StatusBadRequest)
        return
    }

    product := Product{
        ID:        uuid.New().String(),
        Name:      req.Name,
        Description: req.Description,
        Price:     req.Price,
        Stock:     req.Stock,
    }
}
```

```

        CreatedAt:    time.Now(),
        UpdatedAt:    time.Now(),
    }

    if err := h.repo.Create(product); err != nil {
        http.Error(w, "Failed to create product", http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(product)
}

// GetProductByID handles GET /products/{id} requests.
func (h *Handler) GetProductByID(w http.ResponseWriter, r *http.Request) {
    id := chi.URLParam(r, "id")
    if id == "" {
        http.Error(w, "Product ID is required", http.StatusBadRequest)
        return
    }

    product, err := h.repo.GetByID(id)
    if err != nil {
        if errors.Is(err, ErrNotFound) { // Custom error for not found
            http.Error(w, "Product not found", http.StatusNotFound)
            return
        }
        http.Error(w, "Failed to retrieve product", http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(product)
}

// Other CRUD handlers (ListProducts, UpdateProduct, DeleteProduct) would follow a similar pattern.
// ... (omitted for brevity, but would involve decoding, calling repo, encoding response)
...

```

5. Input Validation

Validation should occur early in the handler or service layer. For simple cases, manual checks are sufficient. For complex validation, libraries like ``go-playground/validator`` can be used.

```

...go
// Part of CreateProduct handler (as shown above)

// Basic validation
if req.Name == "" || req.Price <= 0 || req.Stock < 0 {
    http.Error(w, "Name, Price (must be > 0), and Stock (must be >= 0) are required fields.", http.StatusBadRequest)
    return
}

// For more complex validation:
/*
import "github.com/go-playground/validator/v10"
var validate *validator.Validate = validator.New()

type CreateProductRequest struct {
    Name        string `json:"name" validate:"required,min=3,max=100"`
    Description string `json:"description" validate:"omitempty,max=500"`
    Price       float64 `json:"price" validate:"required,gt=0"`
    Stock       int    `json:"stock" validate:"required,gte=0"`
}

// In handler:
if err := validate.Struct(req); err != nil {
    http.Error(w, err.Error(), http.StatusBadRequest) // Or format a better error response
    return
}
*/
...

```

6. Database Layer

Define an interface for the ``ProductRepository`` to abstract the persistence layer. This allows for easy testing with mock implementations and swapping databases (e.g., in-memory, PostgreSQL, MongoDB).

```

...go

```

```
// internal/product/repository/repository.go

package repository

import (
    "errors"
    "sync"
    "your_module_name/internal/product" // Import the product model
)

// ErrNotFound is a sentinel error for when a product is not found.
var ErrNotFound = errors.New("product not found")

// ProductRepository defines the interface for product data operations.
type ProductRepository interface {
    Create(p product.Product) error
    GetByID(id string) (product.Product, error)
    Update(p product.Product) error
    Delete(id string) error
    List() ([]product.Product, error)
}

// internal/product/repository/inmemory.go (Example implementation)

package repository

// InMemoryProductRepository is a simple in-memory implementation of ProductRepository.
type InMemoryProductRepository struct {
    mu      sync.RWMutex
    products map[string]product.Product
}

// NewInMemoryProductRepository creates a new in-memory repository.
func NewInMemoryProductRepository() *InMemoryProductRepository {
    return &InMemoryProductRepository{
        products: make(map[string]product.Product),
    }
}

// Create adds a new product to the map.
func (r *InMemoryProductRepository) Create(p product.Product) error {
    r.mu.Lock()
    defer r.mu.Unlock()
    if _, exists := r.products[p.ID]; exists {
        return errors.New("product with this ID already exists")
    }
    r.products[p.ID] = p
    return nil
}

// GetByID retrieves a product by its ID.
func (r *InMemoryProductRepository) GetByID(id string) (product.Product, error) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    p, ok := r.products[id]
    if !ok {
        return product.Product{}, ErrNotFound // Use custom error
    }
    return p, nil
}

// ... other CRUD methods (Update, Delete, List) for InMemoryProductRepository
...

```

For a PostgreSQL implementation, you would use a driver like ``github.com/jackc/pgx/v5`` and implement the ``ProductRepository`` interface with database queries.

7. Error Handling

Go's idiomatic error handling involves returning ``error`` values. Custom errors (like ``ErrNotFound``) provide more context. Handlers should translate these internal errors into appropriate HTTP status codes and JSON responses.

```
```go
// internal/product/handler.go (Error handling examples)

// In GetProductByID handler:
product, err := h.repo.GetByID(id)
if err != nil {
 if errors.Is(err, ErrNotFound) { // Check for specific custom error
 http.Error(w, "Product not found", http.StatusNotFound)
 }
}

```

```

 return
 }
 log.Printf("Error retrieving product %s: %v", id, err) // Log internal error
 http.Error(w, "Failed to retrieve product", http.StatusInternalServerError)
 return
}

```

```

// For general bad requests:
http.Error(w, "Invalid request payload", http.StatusBadRequest)

```

```

// For internal server errors:
http.Error(w, "Failed to create product", http.StatusInternalServerError)
```

```

8. Project Structure

A common and scalable project structure in Go, following principles like "standard Go project layout" and package cohesion:

```

```
your_module_name/
├── cmd/
│ └── api/
│ └── main.go // Main entry point for the REST API server
├── internal/ // Private application code, not intended for external use
│ └── product/ // Package for product domain logic
│ ├── model.go // Defines Product struct and DTOs
│ ├── handler.go // HTTP handlers for product CRUD
│ └── repository/ // Product data persistence layer
│ ├── repository.go // ProductRepository interface
│ ├── inmemory.go // In-memory implementation of ProductRepository (for dev/testing)
│ └── postgres.go // PostgreSQL implementation (example)
├── pkg/ // Public utility packages (if any, less likely for simple API)
├── go.mod
├── go.sum
└── README.md
```

```

This structure ensures clear separation of concerns:

- * `cmd/`: Contains main applications.
- * `internal/`: Houses application-specific business logic and components.
- * `internal/product/`: Dedicated package for all product-related components.
- * `internal/product/repository/`: Contains the repository interface and its concrete implementations.

Model: gemini-2.5-flash

Token usage (estimated):

Input tokens: 224

Output tokens: 3270

Total tokens: 3494

USER PROMPT:

You are a senior Go developer. I want you to help me plan a REST API for managing products in Go.

Follow the Skeleton of Thought approach:

Step 1: Output only the skeleton of the solution in 6–8 concise bullet points.

The skeleton must cover: data model definition in Go (structs), choice of HTTP framework or net/http, routing, handlers, validations, database layer, error handling, and project structure. Do not expand yet.

Step 2: Expand each bullet point with clear technical details, examples, and Go best practices.

Include sample code snippets in Go (structs, handlers, routes) and considerations about packages (e.g., chi, or net/http), error handling with idiomatic Go, and how to organize the project into packages (handlers, models, db). Use concise and professional language.

The API must implement CRUD operations for products with fields: id, name, description, price, stock.

LLM RESPONSE:

Step 1: Skeleton of Thought

1. **Project Structure:** Implementation of a modular structure (e.g., `cmd/`, `internal/`) to separate entry points from core logic.
2. **Data Models:** Definition of the `Product` struct with Go primitive types and JSON tags for serialization.
3. **HTTP Framework & Routing:** Utilization of `chi` or `net/http` to define RESTful routes (GET, POST, PUT, DELETE).
4. **Database Layer (Repository Pattern):** An interface-driven approach to abstract data storage (SQL/NoSQL) and enable unit testing.
5. **API Handlers:** Logic for processing HTTP requests, decoding JSON, and interacting with the service layer.
6. **Validation:** Integration of struct tags (e.g., `go-playground/validator`) to enforce business rules on input.


```

data.
7. **Error Handling:** Implementation of a centralized, idiomatic error handling strategy to return consistent JSON
error responses.
8. **Configuration & Middleware:** Handling environment variables and cross-cutting concerns like logging and
recovery.

---

### Step 2: Detailed Expansion

#### 1. Project Structure
A standard Go layout ensures scalability.
* `cmd/api/main.go`: Application entry point.
* `internal/product/`: Core domain logic (models, repository, handlers).
* `internal/database/`: DB connection management.
* `pkg/`: Helper packages (e.g., response formatting).

#### 2. Data Models
Define the `Product` struct. Use pointers for optional fields and JSON tags for the API.
```go
type Product struct {
 ID string `json:"id"`
 Name string `json:"name" validate:"required"`
 Description string `json:"description"`
 Price float64 `json:"price" validate:"required,gte=0"`
 Stock int `json:"stock" validate:"gte=0"`
}
```

#### 3. HTTP Framework & Routing
I recommend `go-chi/chi` for its compatibility with `net/http` and its lightweight middleware stack.
```go
func (app *Config) routes() http.Handler {
 r := chi.NewRouter()
 r.Route("/products", func(r chi.Router) {
 r.Post("/", app.CreateProduct)
 r.Get("/", app.GetAllProducts)
 r.Get("/{id}", app.GetProductByID)
 r.Put("/{id}", app.UpdateProduct)
 r.Delete("/{id}", app.DeleteProduct)
 })
 return r
}
```

#### 4. Database Layer (Repository Pattern)
Abstracting the database allows you to switch from PostgreSQL to a Mock DB during testing.
```go
type Repository interface {
 GetByID(id string) (*Product, error)
 Create(p Product) error
 // ... other CRUD methods
}
```

#### 5. API Handlers
Handlers decode the request body and call the repository.
```go
func (h *ProductHandler) CreateProduct(w http.ResponseWriter, r *http.Request) {
 var p Product
 if err := json.NewDecoder(r.Body).Decode(&p); err != nil {
 h.errorJSON(w, err, http.StatusBadRequest)
 return
 }
 // Call repository, handle result...
 h.writeJSON(w, http.StatusCreated, p)
}
```

#### 6. Validation
Use github.com/go-playground/validator/v10 to ensure incoming data is clean before hitting the DB.
```go
var validate = validator.New()

func ValidateStruct(s interface{}) error {
 return validate.Struct(s)
}
```

#### 7. Error Handling

```

Create a standardized JSON error response to ensure the client always receives a predictable format.

```
```go
type JSONResponse struct {
 Error bool `json:"error"`
 Message string `json:"message"`
}

func (app *Config) errorJSON(w http.ResponseWriter, err error, status int) {
 payload := JSONResponse{Error: true, Message: err.Error()}
 app.writeJSON(w, status, payload)
}
```
```

8. Configuration & Middleware

Inject dependencies (DB, Logger) into handlers via a receiver struct. Use `chi/middleware` for logging and panic recovery.

```
```go
type Application struct {
 Repo Repository
 Logger *log.Logger
}

func main() {
 // 1. Load config
 // 2. Init DB
 // 3. Register routes
 // 4. http.ListenAndServe(":8080", app.routes())
}
```
```

****Senior Developer Tip:**** Always use `time.Timeout` on your `http.Server` and use a context-aware database driver (passing `r.Context()`) to ensure your API handles cancellations and timeouts gracefully.

Model: gemini-3-flash-preview

Token usage (estimated):

Input tokens: 224
Output tokens: 1044
Total tokens: 1268
